

Data Preparation/Feature Engineering

Project Title: AI-Powered Crop Disease Detection & Advisory System For Smallholder Farmers In Liberia

Team Members

1. Lydia L Labala
2. Michael Senkao

1. Overview

Data preparation and feature engineering are crucial steps in developing reliable AI models for crop disease detection. High-quality datasets ensure models can generalize well across varying field conditions, while transformations and augmentation help the models learn key visual features of diseased leaves. This section documents the data collection process, cleaning steps, exploratory analysis, transformations, and the initial modeling approach used for building a robust disease detection system targeting cassava, pepper, and rice crops in Liberia.

2. Data Collection

The datasets used in this project were primarily sourced from open-access repositories:

- **PlantVillage Dataset:** A widely used public dataset containing images of healthy and diseased leaves for multiple crops.
- **Cassava Dataset:** Includes diseases such as cassava mosaic and bacterial blight.
- **Pepper Dataset:** Contains healthy and diseased pepper leaves with common fungal infections.
- **Rice Dataset:** Focused on rice blast disease.

Future Data Collection: For real-world relevance in Liberia, we plan to gather images from local farms in collaboration with extension officers. These will help fine-tune the models for contextual performance.

3. Data Cleaning

Raw images often contain inconsistencies that can harm model performance. The following cleaning steps were applied:

1. **Duplicate Removal** – Eliminated repeated images found in the datasets.
2. **Corrupted Image Filtering** – Removed unreadable or damaged image files.
3. **Noise Filtering** – Discarded images with excessive background clutter.
4. **Class Balancing** – Identified imbalanced datasets (e.g., fewer diseased images than healthy ones). Applied oversampling and augmentation to balance classes.
5. **Label Verification** – Cross-checked labels to ensure images were correctly categorized.



4. Exploratory Data Analysis (EDA)

EDA was conducted to better understand dataset structure and potential issues:

- **Class Distribution** – Bar plots were generated to visualize the number of images per category. This highlighted imbalances between healthy and diseased images.
- **Image Dimension Check** – Ensured uniform image dimensions for CNN input (224x224 pixels).
- **Brightness and Contrast Analysis** – Histograms of pixel intensities revealed variation in lighting conditions.
- **Sample Visualization** – Random samples were inspected to confirm label consistency.

5. Data Transformation

The following transformations were applied before feeding images into the models:

- **Resizing** – All images resized to 224x224 pixels.
- **Normalization** – Pixel values scaled to the range [0,1].
- **One-hot Encoding** – Labels converted into categorical vectors.
- **Augmentation** – To improve generalization, images were randomly rotated, flipped, zoomed, and brightness-adjusted.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

Figure 1: Augmentation Pipeline

Model Exploration

1. Model Selection

Two categories of models were explored:

- **CNN Architectures:** ResNet50 and MobileNetV2 for image classification.
- **YOLO (You Only Look Once):** YOLOv5 and YOLOv8 for real-time disease detection.

Rationale: CNNs provide a solid baseline for classification tasks, while YOLO supports detection in real-time mobile applications.

2. Model Training

- **Train-Validation Split:** 80/20.
- **Optimizer:** Adam with learning rate 0.001.
- **Loss Function:** Categorical Crossentropy.
- **Batch Size:** 32.
- **Epochs:** 30–50 with early stopping.

Figure 2: CNN Training

```
: from tensorflow.keras.applications import MobileNetV2
  from tensorflow.keras.models import Model
  from tensorflow.keras.layers import Dense, GlobalAveragePooling2D

  base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(224,224,3))
  x = base_model.output
  x = GlobalAveragePooling2D()(x)
  predictions = Dense(num_classes, activation="softmax")(x)

  model = Model(inputs=base_model.input, outputs=predictions)
  model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
  history = model.fit(train_gen, validation_data=val_gen, epochs=30)
```

3. Model Evaluation

Models were evaluated using:

- **Accuracy** – overall classification performance.
- **Precision & Recall** – reliability for diseased vs healthy classification.
- **F1-score** – balance between precision and recall.
- **Confusion Matrix** – class-level performance insights.
- **ROC Curve** – discriminative ability of classifiers.

Figure 3: Evaluation

```
: from sklearn.metrics import classification_report, confusion_matrix
import numpy as np

Y_pred = model.predict(val_gen)
y_pred = np.argmax(Y_pred, axis=1)

print("Confusion Matrix")
print(confusion_matrix(val_gen.classes, y_pred))
print("Classification Report")
print(classification_report(val_gen.classes, y_pred, target_names=val_gen.class_indices.keys()))
```

4. Code Implementation

The project was implemented using Python with the following libraries:

- **TensorFlow/Keras** – model building and training.
- **OpenCV** – image preprocessing.
- **Matplotlib/Seaborn** – visualization (EDA, evaluation metrics).
- **Scikit-learn** – evaluation (precision, recall, F1, confusion matrix).

```
classes = os.listdir(base_path)
print("Classes available:", classes)

# Count images per class
for cls in classes:
    print(f"{cls}: {len(os.listdir(os.path.join(base_path, cls)))} images")
```

Classes available: ['Pepper_bell_Bacterial_spot', 'Pepper_bell_healthy', 'Potato_Early_blight', 'Potato_healthy', 'Potato_Late_blight', 'Tomato_Bacterial_spot', 'Tomato_Early_blight', 'Tomato_healthy', 'Tomato_Late_blight', 'Tomato_Leaf_Mold', 'Tomato_Septoria_leaf_spot', 'Tomato_Spider_mites_Two_spotted_spider_mite', 'Tomato_Target_Spot', 'Tomato_Tomato_mosaic_virus', 'Tomato_Tomato_YellowLeaf_Curl_Virus']

Pepper_bell_Bacterial_spot: 997 images

Pepper_bell_healthy: 1478 images

Potato_Early_blight: 1000 images

Potato_healthy: 152 images

Potato_Late_blight: 1000 images

Tomato_Bacterial_spot: 2127 images

Tomato_Early_blight: 1000 images

Tomato_healthy: 1591 images

Tomato_Late_blight: 1909 images

Tomato_Leaf_Mold: 952 images

Tomato_Septoria_leaf_spot: 1771 images

Tomato_Spider_mites_Two_spotted_spider_mite: 1676 images

Tomato_Target_Spot: 1404 images

Tomato_Tomato_mosaic_virus: 373 images

Tomato_Tomato_YellowLeaf_Curl_Virus: 3208 images

```
# Load the Pre-trained MobileNetV2 base model (without the top classification layer)
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_HEIGHT, IMG_WIDTH, 3),
    include_top=False,
    weights='imagenet'
)
```

```
# Freeze the base model
base_model.trainable = False

# Add new classification head (custom layers)
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dropout(0.5)(x) #
output = Dense(train_generator.num_classes, activation='softmax')(x)
```

```
# Create the final model
model = Model(inputs=base_model.input, outputs=output)

# Compile the model
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
# Print model summary
model.summary()
```